

THÉORIE DES GRAPHS

Séance 5 : Arbre Couvrant Minimal

Algorithme de Kruskal & Union-Find

L2 - Génie Informatique • Semestre 3

Ing. GBAGUIDI AISSE Alexandre
ESGC VERECHAGUINE A.K. • Cotonou, Bénin

Plan de la séance



Problème

Connecter au moindre coût

10 min



Définitions

Arbre, MST, propriétés

15 min



Propriétés

Coupe et cycle

15 min



Kruskal

Algorithme glouton

20 min



Union-Find

Structure DSU

15 min



Complexité

Kruskal vs Prim

10 min



TD (45 min)

3 exercices MST

Papier



TPE (1h)

Implémenter Kruskal

Code C

Objectifs de la séance

1 Comprendre le problème de l'arbre couvrant minimal

Connecter tous les sommets au moindre coût total
Graphe pondéré non orienté, pas de redondance
Applications : réseaux électriques, câblage, routes

2 Maîtriser l'algorithme de Kruskal

Trier les arêtes, ajouter si pas de cycle
Approche gloutonne avec tri + Union-Find
Complexité $O(E \log E)$ — efficace pour graphes creux

3 Implémenter la structure Union-Find (DSU)

Opérations `make_set`, `find`, `union`
Optimisations : compression de chemin, union par rang
Complexité quasi-constante $O(\alpha(n))$ par opération

Rappel Séance 4 : Dijkstra

Critère	Dijkstra (Séance 4)
Type de graphe	Pondéré (poids ≥ 0)
Structure	File de priorité (Min-Heap)
Principe	Glouton : extraire le minimum, relaxer
Complexité	$O((V+E) \log V)$
Résultat	distance[] et parent[] depuis une source
Limitation	Poids négatifs $\rightarrow$ Bellman-Ford

 **Séance 5 : Nous passons du plus court chemin (Dijkstra) à l'arbre couvrant minimal (Kruskal). Même graphe pondéré, question différente !**

PARTIE 2

Problème Motivant

"Connecter tout le monde au moindre coût"

Connecter au moindre coût

Réseau électrique SBEE

- Connecter 8 villages du Mono
- Chaque câble a un coût (distance)
Objectif : tous connectés, coût MINIMAL
Pas de câble inutile (pas de boucle)

Réseau informatique

- Relier N ordinateurs à ESGC-VAK
- Chaque liaison a un coût (câblage)
Tous doivent pouvoir communiquer
Minimiser le budget total

Formulation du problème

Étant donné un graphe $G = (V, E)$ pondéré non orienté, trouver un sous-ensemble d'arêtes $T \subseteq E$ tel que : (1) T connecte tous les sommets (graphe connexe), (2) T ne contient pas de cycle, (3) La somme des poids de T est MINIMALE. Ce sous-ensemble T forme un Arbre Couvrant Minimal (MST).

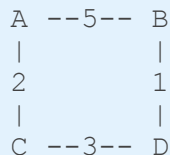
 **Dijkstra = plus court chemin (source → destination). MST = réseau minimal connectant TOUS les sommets. Questions différentes !**

Dijkstra vs MST : deux questions différentes



Dijkstra (Séance 4)

Question : Quel est le chemin le moins cher de A vers B ?

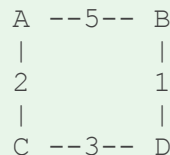


Plus court $A \rightarrow D$: $A \rightarrow C \rightarrow D = 5$ ✓



MST (Séance 5)

Question : Quel réseau connecte TOUT au moindre coût total ?



MST : $A-C(2)$, $B-D(1)$, $C-D(3) = 6$ ✓



Le MST n'est PAS l'arbre des plus courts chemins ! Ce sont deux problèmes fondamentalement différents.

PARTIE 3

Définitions : Arbre Couvrant Minimal

"Qu'est-ce qu'un arbre ? Un arbre couvrant ? Un MST ?"

Qu'est-ce qu'un arbre ?



Définition : Arbre

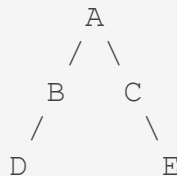
Un arbre est un graphe connexe et acyclique (sans cycle). C'est le graphe connexe avec le MINIMUM d'arêtes possible.



Propriétés d'un arbre à n sommets

- Exactement $n-1$ arêtes
- Connexe (tout sommet accessible)
- Acyclique (aucun cycle)
- Chemin unique entre toute paire
- Supprimer 1 arête $\rightarrow$ déconnexion
- Ajouter 1 arête $\rightarrow$ crée un cycle

Exemple ($n=5$, arêtes=4)



5 sommets, 4 arêtes ✓ Connexe, pas de cycle ✓



Un arbre à n sommets a TOUJOURS exactement $n-1$ arêtes. Si plus $\rightarrow$ cycle. Si moins $\rightarrow$ déconnexion.

Arbre couvrant et MST



Arbre couvrant (Spanning Tree)

Un arbre couvrant d'un graphe G est un sous-graphe qui :

- Inclut TOUS les sommets de G
- Est un arbre (connexe, sans cycle)
- Utilise un sous-ensemble des arêtes de G



Arbre Couvrant Minimal (MST)

Un MST est un arbre couvrant dont la somme totale des poids des arêtes est MINIMALE parmi tous les arbres couvrants possibles.

Exemple : Graphe avec 4 sommets

Graphe complet :

A--1--B A--4--C
B--2--C B--5--D
C--3--D A--6--D

MST (poids total = 6) :

A--1--B (poids 1) ✓

B--2--C (poids 2) ✓

C--3--D (poids 3) ✓

3 arêtes pour 4 sommets (n-1) ✓



Unicité : si tous les poids sont DISTINCTS, le MST est UNIQUE. Sinon, il peut y avoir plusieurs MST de même poids.

PARTIE 4

Propriétés Fondamentales

"Coupe et cycle : pourquoi les algorithmes gloutons marchent"

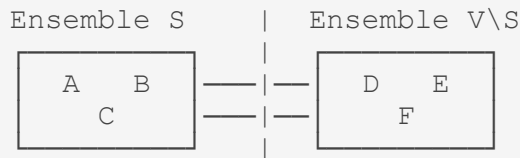
Propriété de coupe (Cut Property)



Propriété de coupe

Soit une coupe $(S, V \setminus S)$ du graphe. L'arête de poids MINIMUM traversant cette coupe appartient à tout MST. (Si les poids sont distincts, elle appartient à l'unique MST.)

Visualisation



L'arête la plus légère qui traverse la coupe est dans le MST !



Pourquoi ça marche ?

Par l'absurde :

Si l'arête min e n'était pas dans le MST T , alors T utilise une autre arête e' plus lourde pour traverser la coupe.

Remplacer e' par e donne un arbre couvrant de poids INFÉRIEUR $\rightarrow T$ n'était pas minimal. Contradiction !



Cette propriété justifie Kruskal ET Prim : à chaque étape, on choisit l'arête la plus légère qui étend notre solution.

Propriété de cycle (Cycle Property)



Propriété de cycle

Dans tout cycle du graphe, l'arête de poids MAXIMUM n'appartient à aucun MST. (Si les poids sont distincts, elle n'appartient pas à l'unique MST.)

Exemple concret

Cycle : A --3-- B --7-- C --2-- A

Poids : 3, 7, 2

Maximum = 7 (arête B-C)

→ L'arête B-C (poids 7) n'est PAS
dans le MST !



Résumé

Propriété de coupe :

→ Arête min d'une coupe

∈ MST ✓

Propriété de cycle :

→ Arête max d'un cycle

∉ MST ✗



Kruskal utilise la propriété de coupe : chaque arête ajoutée est la plus légère traversant la coupe entre composantes.

PARTIE 5

Algorithme de Kruskal

"Trier, puis ajouter si pas de cycle"

Joseph B. Kruskal (1928-2010)



Mathématicien américain

- Né à New York, USA (1928)
- Mathématicien et statisticien
- Chercheur aux Bell Labs & Princeton



L'algorithme de Kruskal

- Publié en 1956 dans le journal *"Proceedings of the AMS"*
- Éléance : tri + détection de cycle
- Approche gloutonne simple et efficace



Applications aujourd'hui

- Conception de réseaux (électrique, télécom, routes)
- Clustering en machine learning
- Analyse d'images (segmentation)
- Bioinformatique (phylogénie)
- Optimisation de circuits VLSI
- Planification urbaine



Kruskal et Dijkstra : même année (1956) ! Deux algorithmes fondamentaux sur les graphes pondérés, publié la même année.

Idée générale de Kruskal



Idée centrale : Algorithme glouton (greedy)

Trier toutes les arêtes par poids croissant. Parcourir les arêtes dans cet ordre : ajouter l'arête au MST si elle ne crée PAS de cycle (i.e. si elle relie deux composantes différentes).

1. Initialisation

Chaque sommet est sa propre composante (forêt de singletons).
MST = ensemble vide.

2. Tri

Trier les E arêtes par poids croissant.
Complexité : $O(E \log E)$

3. Sélection

Pour chaque arête (u,v) :
Si u et v dans composantes $\neq$
→ Ajouter au MST, fusionner

4. Terminaison

S'arrêter quand le MST a exactement $n-1$ arêtes.
Résultat : MST optimal !



Différence avec Dijkstra : Dijkstra choisit le SOMMET le plus proche. Kruskal choisit l'ARÊTE la plus légère.

Algorithme de Kruskal - Pseudo-code

KRUSKAL (Graph G) :

```
// 1. Initialisation
MST = ensemble vide
Pour chaque sommet v:
    MAKE_SET(v)

// 2. Trier les arêtes
Trier E par poids croissant

// 3. Sélection gloutonne
Pour chaque arête (u,v,w) triée:
    Si FIND(u) ≠ FIND(v):
        Ajouter (u,v) à MST
        UNION(u, v)
```

Retourner MST

Points clés

MAKE_SET :

Chaque sommet = 1 ensemble

FIND :

Trouver le représentant

UNION :

Fusionner 2 ensembles

FIND(u) ≠ FIND(v) :

u et v dans des composantes différentes
→ pas de cycle

S'arrête avec n-1 arêtes

Exemple : Dérouler Kruskal

Graphe G (6 sommets, 9 arêtes)

A-B:4 A-C:2 B-C:5
B-D:10 C-D:3 C-E:8
D-E:7 D-F:6 E-F:1

Arêtes triées par poids

E-F:1, A-C:2, C-D:3, A-B:4,
B-C:5, D-F:6, D-E:7, C-E:8,
B-D:10

#	Arête	Poids	Composantes u,v	Action
1	E-F	1	$\{E\} \neq \{F\}$	✓ Acceptée
2	A-C	2	$\{A\} \neq \{C\}$	✓ Acceptée
3	C-D	3	$\{A,C\} \neq \{D\}$	✓ Acceptée
4	A-B	4	$\{A,C,D\} \neq \{B\}$	✓ Acceptée
5	B-C	5	$\{A,B,C,D\} = \text{même}$	✗ Rejetée (cycle)

✓ MST = {E-F, A-C, C-D, A-B, D-F} | Poids total = 1+2+3+4+6 = 16 | 5 arêtes pour 6 sommets (n-1) ✓

PARTIE 6

Structure Union-Find (DSU)

"La clé de l'efficacité de Kruskal"

Pourquoi Union-Find ?

? Problème à résoudre dans Kruskal

À chaque arête (u,v) , on doit : (1) Savoir si u et v sont dans la MÊME composante (pour éviter les cycles), (2) Fusionner deux composantes quand on ajoute une arête. Comment faire ça EFFICACEMENT ?

Approche	Test même composante	Complexité par opération
✗ DFS/BFS à chaque test	Relancer un parcours	$O(V + E)$ par test !
✗ Tableau de composantes	Lookup $O(1)$, mais fusion	$O(V)$ pour fusion
✓ Union-Find (DSU)	FIND en $O(\alpha(n)) \approx O(1)$	$O(\alpha(n)) \approx O(1)$

💡 Exemple concret

Graphe avec $V = 1\,000$ sommets, $E = 5\,000$ arêtes. Sans Union-Find : $O(5\,000 \times 1\,000) = 5\,000\,000$ opérations. Avec Union-Find : $O(5\,000 \times 1) \approx 5\,000$ opérations → 1 000× plus rapide !

Union-Find : 3 opérations

1. MAKE_SET(x)

Créer un ensemble singleton {x}.
 $\text{parent}[x] = x$ (x est sa propre racine)
 $\text{rank}[x] = 0$
Complexité : $O(1)$

```
parent[x] = x  
rank[x] = 0
```

2. FIND(x)

Trouver le REPRÉSENTANT (racine)
de l'ensemble contenant x.
Remonter la chaîne $\text{parent}[]$
jusqu'à trouver $\text{parent}[r] = r$.

```
while parent[x] != x:  
    x = parent[x]  
return x
```

3. UNION(x, y)

Fusionner les ensembles contenant
x et y. Trouver les racines rx, ry.
Si $rx \neq ry \rightarrow$ rattacher l'un à l'autre.

```
rx = FIND(x)  
ry = FIND(y)  
if rx != ry:  
    parent[rx] = ry
```



parent[] est un tableau : parent[i] pointe vers le parent du sommet i. La racine pointe vers elle-même.

Optimisations : deux astuces cruciales



Compression de chemin

Lors de $\text{FIND}(x)$, rattacher chaque nœud visité directement à la racine.

FIND (x) :

```
if parent[x] != x:  
    parent[x] = FIND(parent[x])  
return parent[x]
```

Avant : $A \rightarrow B \rightarrow C \rightarrow D$ (profondeur 3)

Après $\text{FIND}(A)$: $A \rightarrow D, B \rightarrow D, C \rightarrow D$



Union par rang

Lors de UNION, rattacher l'arbre le plus PETIT sous le plus GRAND.

UNION (x, y) :

```
rx = FIND(x), ry = FIND(y)  
if rank[rx] < rank[ry]:  
    parent[rx] = ry  
elif rank[rx] > rank[ry]:  
    parent[ry] = rx  
else:  
    parent[ry] = rx; rank[rx]++
```



Avec les deux optimisations : complexité $O(\alpha(n))$ par opération, où α est la fonction inverse d'Ackermann. En pratique $\alpha(n) \leq 4$ pour tout $n \leq 10^{80} \rightarrow$ quasi-constant $O(1)$!

PARTIE 7

Complexité et Algorithme de Prim

"Kruskal pour les creux, Prim pour les denses"

Complexité détaillée de Kruskal

Tri des arêtes

1

Trier E arêtes par poids croissant
(tri par comparaison)

$O(E \log E)$

Boucle principale

2

Pour chaque arête : 2 FIND + 1 UNION
 E arêtes $\times O(\alpha(V))$ par opération

$O(E \times \alpha(V))$

Total

3

$O(E \log E) + O(E \times \alpha(V))$
 $\alpha(V) \approx \text{constante}$, et $\log E \leq 2 \log V$

$O(E \log E)$
 $\approx O(E \log V)$



Note : $\log E \leq 2 \log V$ car $E \leq V^2$. Donc $O(E \log E) = O(E \log V^2) = O(2E \log V) = O(E \log V)$.

Algorithme de Prim (aperçu)



Idée de Prim : faire "pousser" un arbre depuis un sommet

Partir d'un sommet source. À chaque étape, ajouter l'arête de poids minimum qui connecte un sommet dans l'arbre à un sommet hors de l'arbre. Similaire à Dijkstra : utilise une file de priorité.

Critère	Kruskal	Prim
Approche	Trier les arêtes globalement	Faire pousser depuis 1 sommet
Structure	Union-Find (DSU)	File de priorité (Min-Heap)
Complexité	$O(E \log E)$	$O((V+E) \log V)$
Graphe creux ($E \approx V$)	✓ Meilleur choix	Correct aussi
Graphe dense ($E \approx V^2$)	Plus lent	✓ Meilleur choix
Ressemblance	Tri + ensembles disjoints	Similaire à Dijkstra



En pratique, Kruskal est souvent préféré car plus simple à implémenter. Prim est avantageux pour les graphes très denses.

PARTIE 8

Récapitulatif

"Ce qu'il faut retenir sur le MST"

Points clés à retenir

1

MST = réseau minimal connectant tous les sommets

Arbre couvrant de poids total minimum. Un arbre à n sommets a $n-1$ arêtes.

2

Kruskal = trier les arêtes, ajouter si pas de cycle

Approche gloutonne. Complexité $O(E \log E)$. Idéal pour graphes creux.

3

Union-Find = tester et fusionner les composantes

Avec compression de chemin + union par rang $\rightarrow O(\alpha(n)) \approx O(1)$ par opération.

4

Propriétés de coupe et de cycle justifient l'approche

Arête min d'une coupe $\in$ MST. Arête max d'un cycle $\notin$ MST.

5

Prim = alternative pour graphes denses

Similaire à Dijkstra avec file de priorité. Complexité $O((V+E) \log V)$.

Tableau comparatif : 4 algorithmes

Critère	BFS	DFS	Dijkstra	Kruskal
Type graphe	Non pondéré	Non pondéré	Pondéré (≥ 0)	Pondéré (≥ 0)
Structure	File FIFO	Pile LIFO	File priorité	Tri + Union-Find
Résultat	Plus court (arêtes)	Parcours complet	Plus court (poids)	MST
Complexité	$O(V+E)$	$O(V+E)$	$O((V+E) \log V)$	$O(E \log E)$
Approche	Parcours	Parcours	Glouton (sommet)	Glouton (arête)
Graphe orienté ?	Oui	Oui	Oui	Non (non orienté)



Dijkstra et Kruskal opèrent sur le même type de graphe (pondéré, poids ≥ 0) mais répondent à des questions différentes !